

Terraform 06

Task:

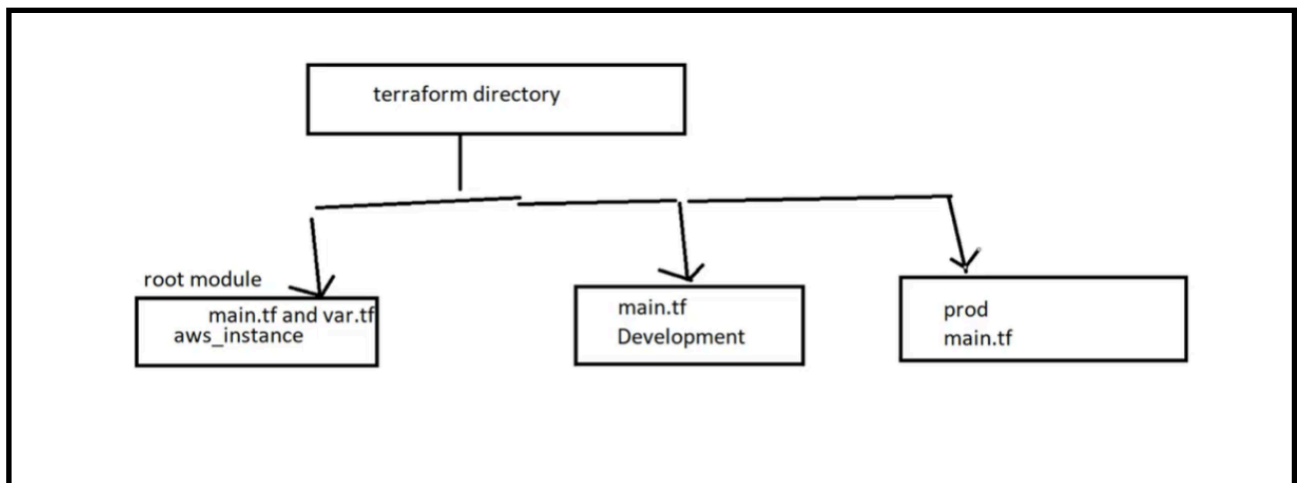
1. Watch the Terraform-06 video.
2. Execute the script shown in the video.

Terraform Modules:

=====

Modules is a collection of configuration files in a directory.

For example if we want to create two ec2 for different environments then i can create a directory for aws-instance configuration.



Example:

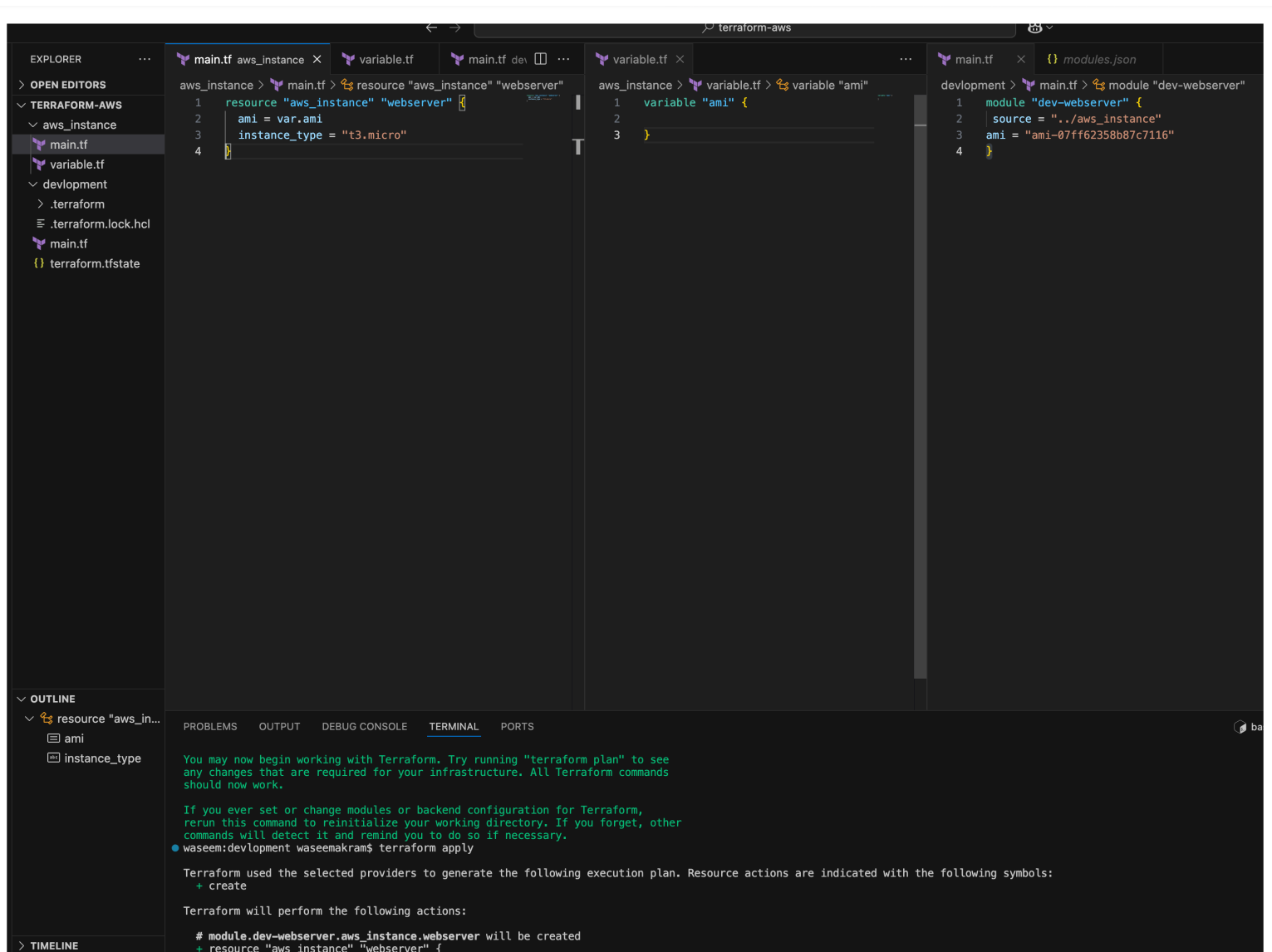
=====

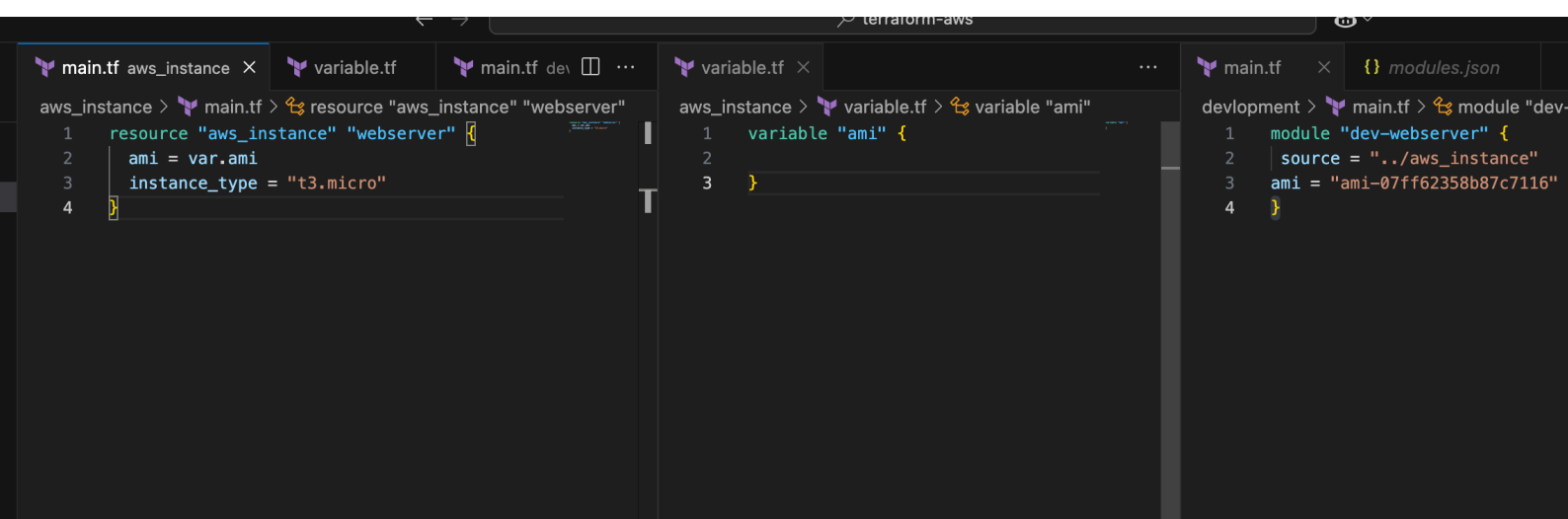
```
resource "aws_instance" "webserver"
ami = "var.ami"
instance_type = "t2.micro"
}
```

now let me create a directory for dev under root directory.

example:

```
module "dev-webserver" {
  source = "../aws_instance"
  ami = "ami-02f3f602d23f1659d"
}
```





--> go to development directory and -> terraform init

--> as development dir is having module in main.tf

```
waseem:terraform-aws waseemakram$ ls
aws_instance  development
waseem:terraform-aws waseemakram$ cd development/
waseem:development waseemakram$ ls
main.tf
```

--> terraform init

--> terraform apply

```
Terraform used the selected providers to generate the following execution plan. Resource
+ create

Terraform will perform the following actions:

# module.dev-webserver.aws_instance.webserver will be created
+ resource "aws_instance" "webserver" {
+   ami                                = "ami-07ff62358b87c7116"
+   arn                                = (known after apply)
+   associate_public_ip_address       = (known after apply)
+   availability_zone                  = (known after apply)
+   disable_api_stop                   = (known after apply)
+   disable_api_termination            = (known after apply)
+   ebs_optimized                      = (known after apply)
+   enable_primary_ipv6                = (known after apply)
```

```
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

module.dev-webserver.aws_instance.webserver: Creating...
module.dev-webserver.aws_instance.webserver: Still creating... [00m10s elapsed]
module.dev-webserver.aws_instance.webserver: Creation complete after 19s [id=i-0cf065b19e396b5c1]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
○ waseem:development waseemakram$
```

--> now check your ec2 instance in aws

☐		i-0cf065b19e396b5c1	✔ Running  	t3.micro	 Initializing
☐	tomcat-ec2	i-06ebaafdc9d1343c9	⊖ Stopped  	t3.micro	–
☐	jenkins-slave	i-0bf14b0eb013d170a	⊖ Stopped  	t3.micro	–

--> modules are used when if we want same configuration from different environments .

Terraform workspaces:

=====

Workspace can be used if we have multiple project/ environments using the same configuration.

--> Workspace commands:

=====

terraform workspace new projectA --> to create a workspace names projectA

terraform workspace list --> to list the workspaces available

terraform workspace select projectA --> to switch to specific workspace

```
waseemakram@waseem development % terraform workspace show
default
waseemakram@waseem development % terraform workspace list
* default

waseemakram@waseem development % terraform workspace new projectA
Created and switched to workspace "projectA"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
waseemakram@waseem development % terraform workspace list
default
* projectA

waseemakram@waseem development %
```

```
waseemakram@waseem development % terraform workspace project projectA
Usage: terraform [global options] workspace

    new, list, show, select and delete Terraform workspaces.

Subcommands:
  delete  Delete a workspace
  list    List Workspaces
  new     Create a new workspace
  select  Select a workspace
  show    Show the name of the current workspace
waseemakram@waseem development % terraform workspace select projectA
waseemakram@waseem development % terraform workspace list
default
* projectA

waseemakram@waseem development % terraform workspace show
projectA
waseemakram@waseem development % terraform workspace select default
Switched to workspace "default".
waseemakram@waseem development % terraform workspace list
* default
projectA

waseemakram@waseem development %
```

Example:

main.tf:

=====

```
resource "aws+instance" "webserver"
```

```
ami = "var.ami"
```

```
instance_type = "t2.micro"
```

```
}
```

variable.tf:

=====

```
variable "ami" {
```

```
type = map
```

```
default = {
```

```
  "ProjectA" = "ami-02f3f602d23f1659d"
```

```
  "ProjectB" = "ami-02f3f602d23f1659d"
```

```
}
```

```
}
```

when using workspaces state files will be created in the main directory

under a folder "terraform.tfstate.d"

The screenshot shows a VS Code editor with several tabs open: `main.tf` (aws_instance), `main.tf development`, `main.tf .../production`, `variable.tf`, and `modules.json`. The `main.tf` tab is active, showing a Terraform configuration for an AWS instance named "webserver". The configuration includes a provider for "aws" with region "us-east-1", and a resource "aws_instance" with attributes `ami = var.ami`, `instance_type = "t3.micro"`, and a tag `name-server`. The `variable.tf` tab is also visible, showing a variable "ami" with a default value of "ami-07ff62358b87c7116".

The terminal window at the bottom shows the output of the `terraform apply` command. It lists the resources to be added, including `capacity_reservation_specification`, `cpu_options`, `ebs_block_device`, `enclave_options`, `ephemeral_block_device`, `instance_market_options`, `maintenance_options`, `metadata_options`, `network_interface`, `primary_network_interface`, `private_dns_name_options`, and `root_block_device`. The plan shows 1 to add, 0 to change, and 0 to destroy. The user is prompted to confirm the actions, and they enter "yes". The terminal then shows the creation of the `aws_instance.webserver` resource, which is completed after 20 seconds.

```
aws_instance > main.tf > resource "aws_instance" "webserver"
1 provider "aws" {
2   region = "us-east-1"
3 }
4 resource "aws_instance" "webserver" {
5   ami = var.ami ["projectA"]
6   instance_type = "t3.micro"
7   tags = {
8     Name = "name-server"
9   }
10 }
```

```
aws_instance > variable.tf > variable "ami"
1 variable "ami" {
2   type = map(string)
3   default = {
4     "projectA" = "ami-07ff62358b87c7116"
5   }
6 }
7 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

+ capacity_reservation_specification (known after apply)
+ cpu_options (known after apply)
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions in workspace "projectA"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver: Creation complete after 20s [id=i-053f29d9200f76332]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
● waseemakram@waseem aws_instance % ls
main.tf          production      terraform.tfstate.d  variable.tf
● waseemakram@waseem aws_instance % cd terraform.tfstate.d
● waseemakram@waseem terraform.tfstate.d % ls
projectA
○ waseemakram@waseem terraform.tfstate.d %
```

--> terraform init
--> terraform apply

```
● waseemakram@waseem aws_instance % ls
main.tf          production      terraform.tfstate.d  variable.tf
● waseemakram@waseem aws_instance % cd terraform.tfstate.d
● waseemakram@waseem terraform.tfstate.d % ls
projectA
○ waseemakram@waseem terraform.tfstate.d %
```

--> now look at aws account

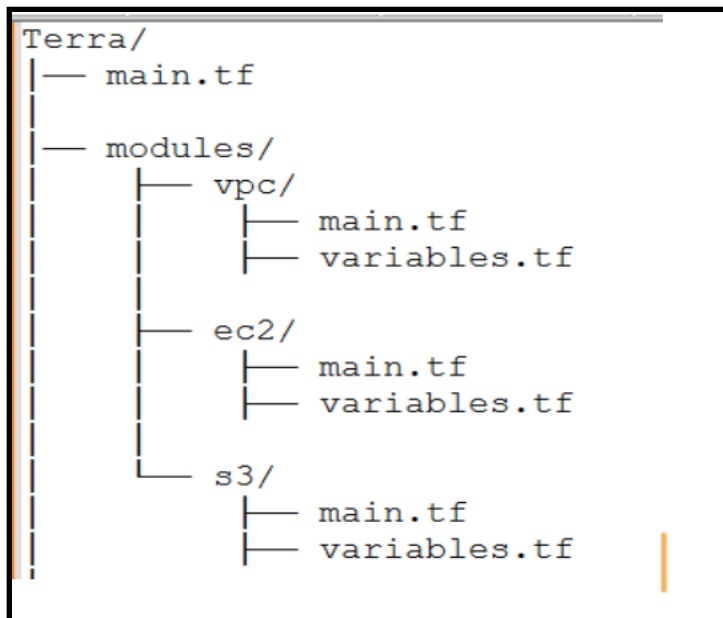
☐	name-server	i-053f29d9200f76332	Running	t3.micro	Initializing	View alarms +	us-east-1c	ec2-54-221-151-211.co...
☐	sonarqube	i-01cb0fac0199e8107	Stopped	m7i-flex.large	-	View alarms +	us-east-1b	-
☐	sonarqube2	i-08b064238eac3cac0	Stopped	m7i-flex.large	-	View alarms +	us-east-1b	-
☐	nexus	i-0dc2978b8f9efc9f4	Stopped	m7i-flex.large	-	View alarms +	us-east-1b	-

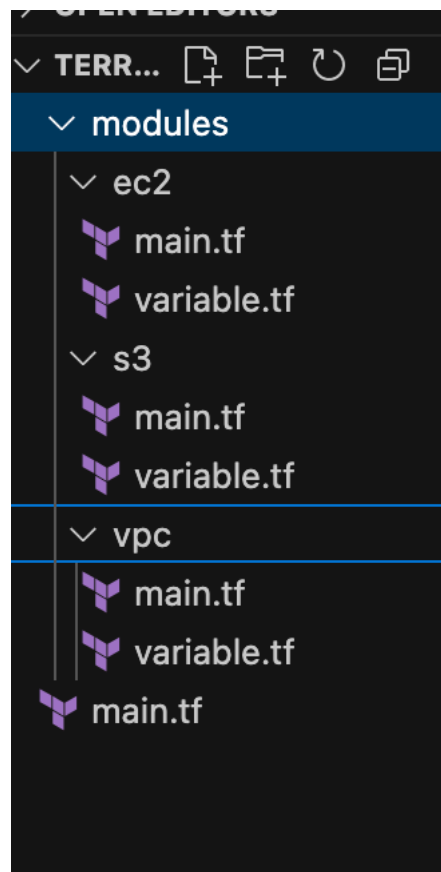
Task: 3

3.Provision **EC2, S3, and VPC** using **Terraform modules**.

structure step by step with re-usable modules.

provision **EC2, S3, and VPC** using **Terraform modules**.





Now in your main.tf

TERRAFORM-AWS/main.tf

EXPLORER

...

> OPEN EDITORS

TERRAFORM-AWS

> .terraform

< modules

< ec2

main.tf

variable.tf

< s3

main.tf

variable.tf

< vpc

main.tf

variable.tf

≡ .terraform.lock.hcl

main.tf

{ } terraform.tfstate

≡ terraform.tfstate.ba...

main.tf ./ X

main.tf .../vpc

variable.tf .../vpc

main.

main.tf > ...

1 provider "aws" {

2 | region = "us-east-1"

3 }

4

5

6

7 module "vpc" {

8 | source = "./modules/vpc"

9 | vpc_cidr = "10.0.0.0/16"

10 | public_subnet_cidr = "10.0.1.0/24"

11 }

12

13

14 module "s3" {

15 | source = "./modules/s3"

16 | bucket_name = "waseem-terraform-demo-bucket-12345"

17 }

18

19 module "ec2" {

20 | source = "./modules/ec2"

21 | instance_type = "t3.micro"

22 | subnet_id = module.vpc.public_subnet_id

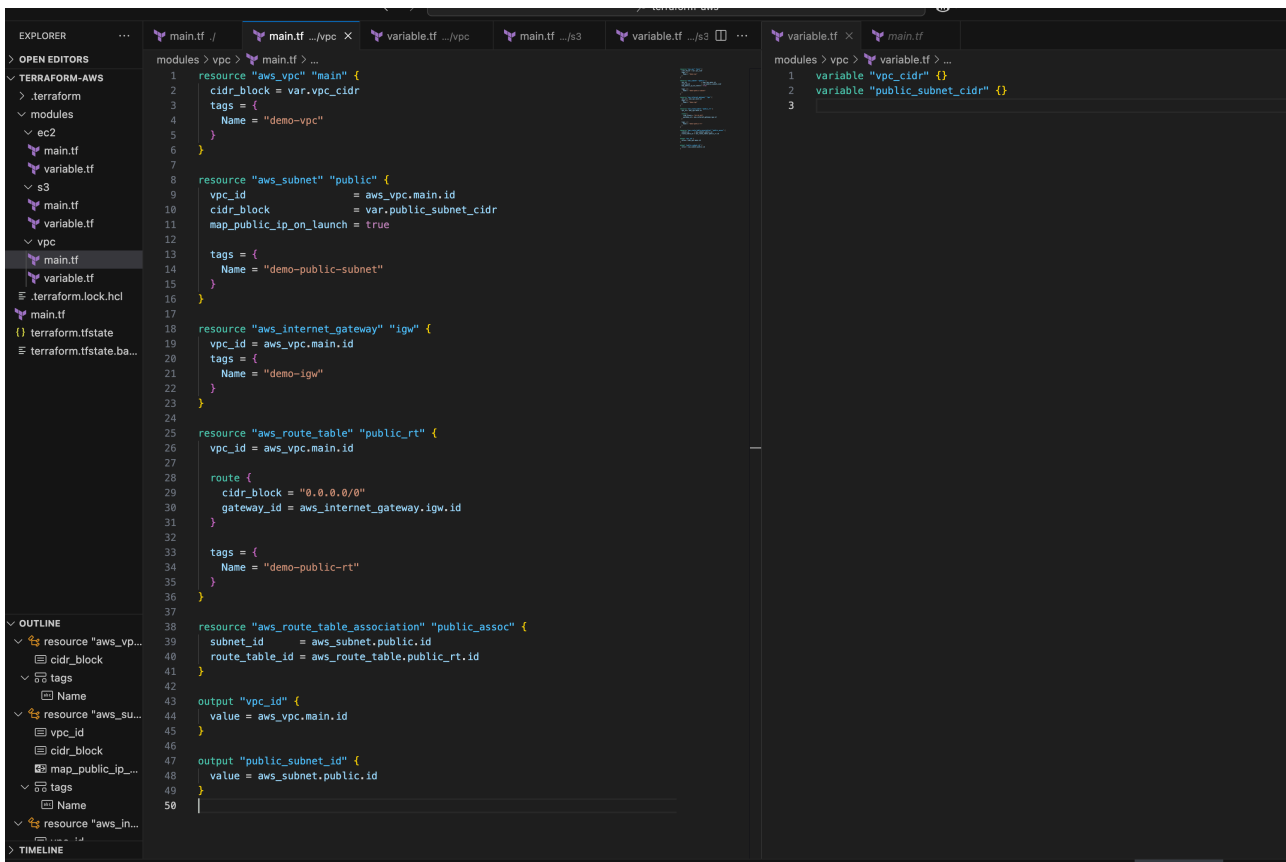
23 | vpc_id = module.vpc.vpc_id

24 }

25

26

MODULE: VPC



```
1 resource "aws_vpc" "main" {
2   cidr_block = var.vpc_cidr
3   tags = {
4     Name = "demo-vpc"
5   }
6 }
7
8 resource "aws_subnet" "public" {
9   vpc_id = aws_vpc.main.id
10  cidr_block = var.public_subnet_cidr
11  map_public_ip_on_launch = true
12
13  tags = {
14    Name = "demo-public-subnet"
15  }
16 }
17
18 resource "aws_internet_gateway" "igw" {
19   vpc_id = aws_vpc.main.id
20   tags = {
21     Name = "demo-igw"
22   }
23 }
24
25 resource "aws_route_table" "public_rt" {
26   vpc_id = aws_vpc.main.id
27
28   route {
29     cidr_block = "0.0.0.0/0"
30     gateway_id = aws_internet_gateway.igw.id
31   }
32
33   tags = {
34     Name = "demo-public-rt"
35   }
36 }
37
38 resource "aws_route_table_association" "public_assoc" {
39   subnet_id = aws_subnet.public.id
40   route_table_id = aws_route_table.public_rt.id
41 }
42
43 output "vpc_id" {
44   value = aws_vpc.main.id
45 }
46
47 output "public_subnet_id" {
48   value = aws_subnet.public.id
49 }
50
```

```
1 variable "vpc_cidr" {}
2 variable "public_subnet_cidr" {}
3
```

--> modules/vpc/variable.tf



```
variable.tf × main.tf
modules > vpc > variable.tf > ...
1 variable "vpc_cidr" {}
2 variable "public_subnet_cidr" {}
3
```

--> modules/vpc/main.tf

```
modules > vpc > main.tf > ...
1  resource "aws_vpc" "main" {
2      cidr_block = var.vpc_cidr
3      tags = {
4          Name = "demo-vpc"
5      }
6  }
7
8  resource "aws_subnet" "public" {
9      vpc_id = aws_vpc.main.id
10     cidr_block = var.public_subnet_cidr
11     map_public_ip_on_launch = true
12
13     tags = {
14         Name = "demo-public-subnet"
15     }
16 }
17
18 resource "aws_internet_gateway" "igw" {
19     vpc_id = aws_vpc.main.id
20     tags = {
21         Name = "demo-igw"
22     }
23 }
24
25 resource "aws_route_table" "public_rt" {
26     vpc_id = aws_vpc.main.id
27
28     route {
29         cidr_block = "0.0.0.0/0"
30         gateway_id = aws_internet_gateway.igw.id
31     }
32
33     tags = {
34         Name = "demo-public-rt"
35     }
36 }
37
38 resource "aws_route_table_association" "public_assoc" {
39     subnet_id = aws_subnet.public.id
40     route_table_id = aws_route_table.public_rt.id
41 }
42
43 output "vpc_id" {
44     value = aws_vpc.main.id
45 }
46
47 output "public_subnet_id" {
48     value = aws_subnet.public.id
49 }
50 |
```

MODULE: S3

```
modules > s3 > main.tf > resource "aws_s3_bucket" "bucket" > tags
1 resource "aws_s3_bucket" "bucket" {
2   bucket = var.bucket_name
3
4   tags = {
5     Name = "demo-s3"
6   }
7 }
8

modules > s3 > variable.tf > ...
1 variable "bucket_name" {}
2
```

modules/s3/variable.tf

```
modules > s3 > variable.tf > ...
1 variable "bucket_name" {}
2
```

modules/s3/main.tf

```
modules > s3 > main.tf > resource "aws_s3_bucket" "bucket" > t
1 resource "aws_s3_bucket" "bucket" {
2   bucket = var.bucket_name
3
4   tags = {
5     Name = "demo-s3"
6   }
7 }
8
```

MODULE: EC2

—>modules/ec2/main.tf
>modules/ec2/variable.tf

```
modules > ec2 > main.tf > resource "aws_security_group" "ec2_sg" > egress
1 resource "aws_security_group" "ec2_sg" {
2   name = "demo-ec2-sg"
3   vpc_id = var.vpc_id
4
5   ingress {
6     from_port = 22
7     to_port = 22
8     protocol = "tcp"
9     cidr_blocks = ["0.0.0.0/0"]
10  }
11
12  egress {
13    from_port = 0
14    to_port = 0
15    protocol = "-1"
16    cidr_blocks = ["0.0.0.0/0"]
17  }
18 }
19
20 resource "aws_instance" "demo" {
21   ami = "ami-07ff62358b87c7116" # Amazon Linux 2 (example)
22   instance_type = var.instance_type
23   subnet_id = var.subnet_id
24   vpc_security_group_ids = [aws_security_group.ec2_sg.id]
25
26   tags = {
27     Name = "demo-ec2"
28   }
29 }
30
```

```
modules > ec2 > variable.tf > variable "instance_type"
1 variable "instance_type" {}
2 variable "subnet_id" {}
3 variable "vpc_id" {}
4
```

—> above main.tf is with security group
—> below without security group

```
~/Desktop/terraform-aws/modules/vpc/main.tf
1
2
3 resource "aws_instance" "demo" {
4   ami = "ami-07ff62358b87c7116" # Amazon Linux 2 (example)
5   instance_type = var.instance_type
6   subnet_id = var.subnet_id
7
8
9   tags = {
10     Name = "demo-ec2"
11   }
12 }
13
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Plan: 8 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

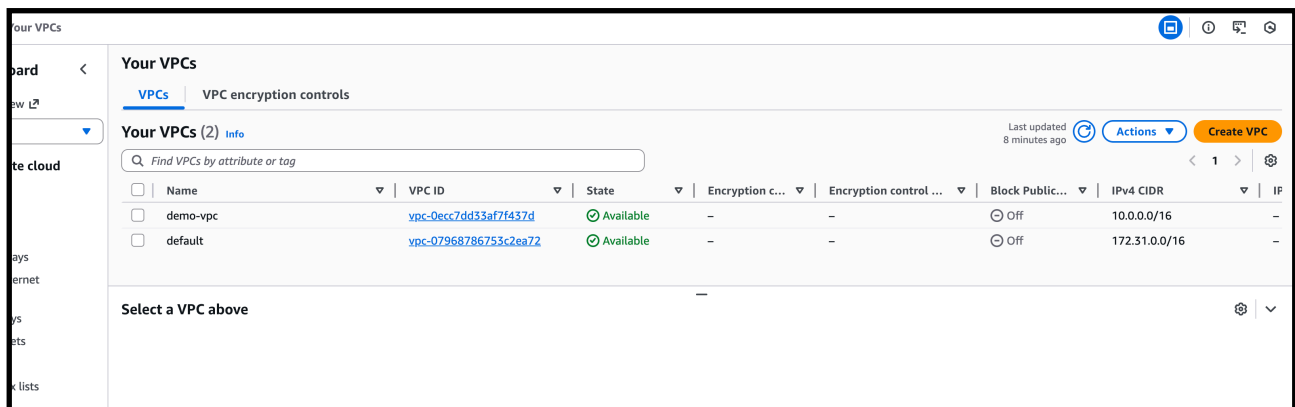
module.vpc.aws_vpc.main: Creating...
module.s3.aws_s3_bucket.bucket: Creating...
module.vpc.aws_vpc.main: Creation complete after 4s [id=vpc-0ecc7dd33af7f437d]
module.vpc.aws_internet_gateway.igw: Creating...
module.vpc.aws_subnet.public: Creating...
module.ec2.aws_security_group.ec2_sg: Creating...
module.s3.aws_s3_bucket.bucket: Creation complete after 5s [id=waseem-terraform-demo-bucket-12]
module.vpc.aws_internet_gateway.igw: Creation complete after 2s [id=igw-005b66e12ac81ec48]
module.vpc.aws_route_table.public_rt: Creating...
module.vpc.aws_route_table.public_rt: Creation complete after 2s [id=rtb-0c8b63788e1d37e24]
module.ec2.aws_security_group.ec2_sg: Creation complete after 6s [id=sg-0783752502f27e3df]
module.vpc.aws_subnet.public: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public: Creation complete after 13s [id=subnet-0c86360571f8a1981]
module.vpc.aws_route_table_association.public_assoc: Creating...
module.ec2.aws_instance.demo: Creating...
module.vpc.aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0]
module.ec2.aws_instance.demo: Still creating... [00m10s elapsed]
module.ec2.aws_instance.demo: Creation complete after 15s [id=i-010b606495e4fa7aa]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.
```

Run Commands

Inside root folder TERRAFORM-AWS/

```
terraform init
terraform plan
terraform apply
```



—> now lets check the aws account

—-> **vpc**

—> demo-vpc created

Subnets:

☒	demo-public-subnet	subnet-0381208e78aeba95f	✔ Available	vpc-04d625ffa87d0c4a6	⊖ Off	10.0.1.0/24
☐	pubsubnet	subnet-071d209e29a1a1f82	✔ Available	vpc-07968786753c2ea72 defa...	⊖ Off	172.31.0.0/20
☐	-	subnet-0c303e20528327bd8	✔ Available	vpc-07968786753c2ea72 defa...	⊖ Off	172.31.48.0/20
☐	-	subnet-0d8a1eba2c0b2c443	✔ Available	vpc-07968786753c2ea72 defa...	⊖ Off	172.31.32.0/20
☐	-	subnet-0f208e7a1554613af	✔ Available	vpc-07968786753c2ea72 defa...	⊖ Off	172.31.80.0/20
☒	demo-public-subnet	subnet-0c86360571f8a1981	✔ Available	vpc-0ecc7dd33af7f437d demo...	⊖ Off	10.0.1.0/24

Subnets: [subnet-0381208e78aeba95f](#), [subnet-0c86360571f8a1981](#)

—> vpc —> subnets also created

—> demo-public-subnet

—-> demo-private-subnet

Igw

☐	demo-igw	igw-005b66e12ac81ec48	✔ Attached	vpc-0ecc7dd33af7f437d demo-vpc	207662791773
☐	default-vpc	igw-05d93d6a24c45086c	✔ Attached	vpc-07968786753c2ea72 default	207662791773

Route table:

☐	default-rt	rtb-0b5d29d80613137e7	-	-	Yes	vpc-07968786
☐	demo-public-rt 	rtb-0c8b63788e1d37e24	subnet-0c86360571f8a1...	-	No	vpc-0ecc7dd33

Now s3:

☐	waseem-terraform-demo-bucket-12345	US East (N. Virginia) us-east-1	January 22, 2026, 18:15:12 (UTC+05:30)	permissions that all AWS accounts.
-----------------------	--	---------------------------------	--	------------------------------------

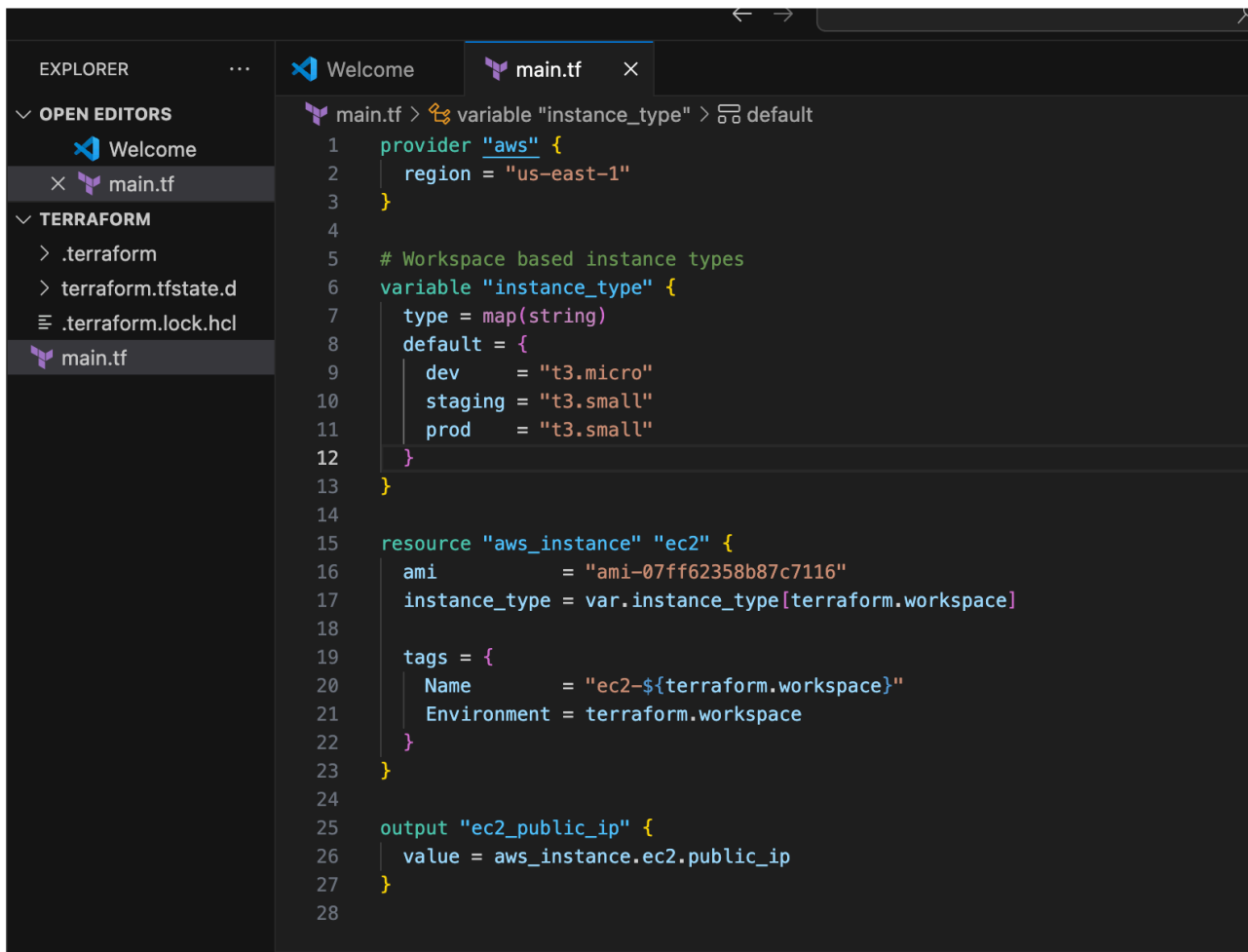
Ec2:

☐	demo-ec2	i-010b606495e4fa7aa	 Running  	t3.micro	 Initializing	View alarms 	us-east-1d	-	50.1
☐	jenkins-master	i-0fe26bc99f7f8361b	 Stopped  	m7i-flex.large	-	View alarms 	us-east-1f	-	-
☐	terraform-ec2	i-0f6bbaafdc0d1747e0	 Canceled  	t3.micro	 Initializing	View alarms 	us-east-1a	-	-

Task: 4

4.Provision **EC2 for 3 different environments** (Dev, Staging, and Prod) using **Terraform workspaces**.

—> create a directory —> create main.tf



```
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 # Workspace based instance types
6 variable "instance_type" {
7   type = map(string)
8   default = {
9     dev    = "t3.micro"
10    staging = "t3.small"
11    prod   = "t3.small"
12  }
13 }
14
15 resource "aws_instance" "ec2" {
16   ami           = "ami-07ff62358b87c7116"
17   instance_type = var.instance_type[terraform.workspace]
18
19   tags = {
20     Name        = "ec2-${terraform.workspace}"
21     Environment = terraform.workspace
22   }
23 }
24
25 output "ec2_public_ip" {
26   value = aws_instance.ec2.public_ip
27 }
28
```

Add above details in main.tf

EC2 for 3 different environments (Dev, Staging, and Prod)

In terminal —> create environments —>

Command for creating new environments

—> terraform workspace new dev

```
waseemakram@waseem terraform % terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
waseemakram@waseem terraform % terraform apply
data.aws_vpc.default: Reading...
data.aws_vpc.default: Read complete after 2s [id=vpc-07968786753c2ea72]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
+ create

Terraform will perform the following actions:

# aws_instance.app will be created
```

```
commands will detect it and remind you to do so if necessary.
waseemakram@waseem terraform % terraform workspace list
default
* dev

waseemakram@waseem terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

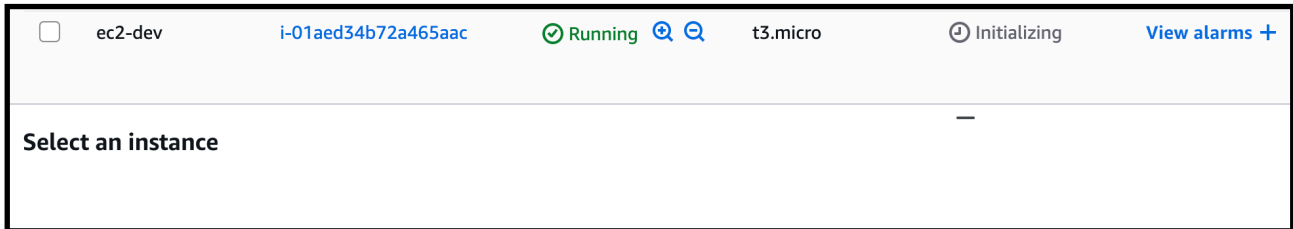
Terraform will perform the following actions:

# aws_instance.ec2 will be created
+ resource "aws_instance" "ec2" {
  + ami                    = "ami-07ff62358b87c7116"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + cpu_core_count        = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop       = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + get_password_data      = false
  + host_id                = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
```

—-> after that

—-> terraform apply

--> check your aws ec2



similarly for staging env

--> terraform workspace new staging

--> terraform workspace list

--> terraform apply

```
● waseemakram@waseem terraform % terraform workspace new staging
Created and switched to workspace "staging"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
● waseemakram@waseem terraform % terraform workspace list
  default
  dev
* staging

● waseemakram@waseem terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.ec2 will be created
+ resource "aws_instance" "ec2" {
  + ami                  = "ami-07ff62358b87c7116"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
```

Ln 12, Col 4 Space

→ now check aws ec2

☐	ec2-staging	i-0672a8ff474067d24	 Running  	t3.small	 Initializing	View alarms +	us-east-1c
☐	demo-ec2	i-010b606495e4fa7aa	 Terminated  	t3.micro	-	View alarms +	us-east-1d

--> and again similarly for prod env

```
waseemakram@waseem terraform % terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.

waseemakram@waseem terraform % terraform workspace list
  default
  dev
* prod
  staging

waseemakram@waseem terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.ec2 will be created
+ resource "aws_instance" "ec2" {
  + ami              = "ami-07ff62358b87c7116"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count    = (known after apply)
```

```
--> terraform workspace new prod
```

```
--> terraform workspace list
```

—> terrafrom apply

☐	ec2-dev	i-01aed34b72a465aac	Terminated	t3.micro	–	View alarms +	us-east-1f	–	–
☐	ec2-prod	i-0f8833bb2dfb924d7	Running	t3.small	Initializing	View alarms +	us-east-1f	ec2-44-220-44-196.co...	44.220.44.15
select an instance									